

AG3 - Actividad Guiada 3

Nombre: Irune Urrutia Andres

Link:

[https://colab.research.google.com/drive/1UTpoX3d8EZkqbV94GOJl_egdpz1FES55#scrollTo=H-ngpjJuBK1q\\$0](https://colab.research.google.com/drive/1UTpoX3d8EZkqbV94GOJl_egdpz1FES55#scrollTo=H-ngpjJuBK1q$0)

Github: [https://github.com/iruneurrutia/O3MIAR_ALGORITMOS_ACTIVIDADES\\$0](https://github.com/iruneurrutia/O3MIAR_ALGORITMOS_ACTIVIDADES$0)

✓ Instalación de librerías

```
!pip install requests
import urllib.request
```

```
!pip install tsplib95
import tsplib95
```

```
Requirement already satisfied: requests in /usr/local/lib/python3.13
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.13
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.13
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.13
Requirement already satisfied: certifi<2017.4.17 in /usr/local/lib/python3.13
Collecting tsplib95
```

```
  Downloading tsplib95-0.7.1-py2.py3-none-any.whl.metadata (6.3 kB)
Requirement already satisfied: Click>=6.0 in /usr/local/lib/python3.13
Collecting Deprecated~1.2.9 (from tsplib95)
```

```
  Downloading Deprecated-1.2.18-py2.py3-none-any.whl.metadata (5.7 kB)
Collecting networkx~2.1 (from tsplib95)
```

```
  Downloading networkx-2.8.8-py3-none-any.whl.metadata (5.1 kB)
Collecting tabulate~0.8.7 (from tsplib95)
```

```
  Downloading tabulate-0.8.10-py3-none-any.whl.metadata (25 kB)
Collecting wrapt<2,>=1.10 (from Deprecated~1.2.9->tsplib95)
```

```
  Downloading wrapt-1.17.3-cp313-cp313-manylinux1_x86_64.manylinux_2_28
  Downloading tsplib95-0.7.1-py2.py3-none-any.whl (25 kB)
```

```
  Downloading Deprecated-1.2.18-py2.py3-none-any.whl (10.0 kB)
  Downloading networkx-2.8.8-py3-none-any.whl (2.0 MB)
```

```
 2.0/2.0 MB 26.9 MB/s eta 0:00
  Downloading tabulate-0.8.10-py3-none-any.whl (29 kB)
```

```
  Downloading wrapt-1.17.3-cp313-cp313-manylinux1_x86_64.manylinux_2_28
 88.1/88.1 kB 7.3 MB/s eta 0:00
```

```
Installing collected packages: wrapt, tabulate, networkx, Deprecated
```

```
  Attempting uninstall: wrapt
```

```
    Found existing installation: wrapt 2.4.0
```

```
  Uninstalling wrapt-2.4.0:
```

```
    Successfully uninstalled wrapt-2.4.0
```

```
  Attempting uninstall: tabulate
```

```
    Found existing installation: tabulate 0.9.0
```

```
  Uninstalling tabulate-0.9.0:
```

```
    Successfully uninstalled tabulate-0.9.0
```

```
  Attempting uninstall: networkx
```

```
    Found existing installation: networkx 3.6.1
```

```
  Uninstalling networkx-3.6.1:
```

```
    Successfully uninstalled networkx-3.6.1
```

```
ERROR: pip's dependency resolver does not currently take into account
momepy 0.11.0 requires networkx>=3.2, but you have networkx 2.8.8 wh
spopt 0.7.0 requires networkx>=3.2, but you have networkx 2.8.8 whic
bigframes 2.48.0 requires tabulate>=0.9, but you have tabulate 0.8.1
scikit-image 0.25.2 requires networkx>=3.0, but you have networkx 2.8
mapclassify 2.10.0 requires networkx>=3.2, but you have networkx 2.8
Successfully installed Deprecated-1.2.18 networkx-2.8.8 tabulate-0.8
```

```
import random    #Libreria para generar numeros y listas aleatorias
import copy      #Permite hacer copias de objetos en python: listas,
```

✓ Carga de datos del problema

```
#Librerias y carga del problema

#http://elib.zib.de/pub/mp-testdata/tsp/tsplib/
#Documentacion :
# https://wwwproxy.iwr.uni-heidelberg.de/groups/comopt/software/T
# https://tsplib95.readthedocs.io/usage.html
# https://tsplib95.readthedocs.io/modules.html#module-tsplib95.mc

#Matriz de distancias
file = "swiss42.tsp" ;
urllib.request.urlretrieve("https://raw.githubusercontent.com/mastoc

#Objeto de tsplib95 para nuestro problema problema
problem = tsplib95.load(file)

#Nodos
Nodos = list(problem.get_nodes())

#Aristas
Aristas = list(problem.get_edges())
print(Aristas)

#Coordenadas(si estan disponibles en el fichero)
problem.get_display(1)

#Distancia
problem.get_weight(1, 2)

[(0, 0), (0, 1), (0, 2), (0, 3), (0, 4), (0, 5), (0, 6), (0, 7), (0,
34
```

✓ Funciones de la Actividad Guiada 3

```

#Funciones de la Actividad Guiada 3
#Se genera una solucion aleatoria con comienzo en en el nodo 0
def crear_solucion(Nodos):
    solucion = [Nodos[0]]
    for n in Nodos[1:] :
        solucion = solucion + [random.choice(list(set(Nodos) - set(solucion)))]
    return solucion

#Devuelve la distancia entre dos nodos
def distancia(a,b, problem):
    return problem.get_weight(a,b)

#Devuelve la distancia total de una trayectoria/solucion
def distancia_total(solucion, problem):
    distancia_total = 0
    for i in range(len(solucion)-1):
        distancia_total += distancia(solucion[i] ,solucion[i+1] , problem)
    return distancia_total + distancia(solucion[len(solucion)-1] ,solucion[0] , problem)

```

✓ Funciones Auxiliares

```

#Genera una poblacion inicial de soluciones de tamaño N.
# Puede ser válida la solución aleatoria de la AG3: crear_solucion()
def generar_poblacion(Nodos,N):
    #

```

File ["/tmp/ipykernel_4094/3021230068.py"](#), line 4

^

_IncompleteInputError: incomplete input

Pasos siguientes: [Explicar error](#)

```

#Evalua la población y devuelve el mejor individuo
def Evaluar_Poblacion(poblacion, problem):
    #...

```

```

#Funcion de cruce. Recibe una poblacion(lista de soluciones) y devuelve
# Todos los individuos de la población son seleccionados para el cruce
# Podría aplicarse un proceso previo de selección para elegir los individuos
def Cruzar(poblacion,mutacion, problem):
    #...

```

```
#Funcion para generar hijos a partir de 2 padres:
# Se elige el metodo de 1-punto de corte pero es posible usar otros
def Descendencia(padres, problem,mutacion):
    #...
```

```
#Para el operador de cruce 1-punto los hijos generados no son soluc
def Factibilizar(solucion,problem):
    #...
```

```
#Funcion de mutación. Se eligen dos nodos y se intercambia. Se podr
# Se hace mutaciones mutacion% de las veces
def Mutar(solucion,mutacion):
    #
```

```
#Funcion de seleccion de la población. Recibe como parametro una pc
# devuelve una poblacion a la que se ha eliminado individuos poco a
#Se tiene en cuenta el porcentaje elitismo pasado como parametro
# Para los individuos que no son de la elite podríamos usar una sel
def Seleccionar(problem,poblacion, N, elitismo):
```

✓ Proceso Principal

```
#Funcion principal del algoritmo genetico
#####3
def algoritmo_genetico(problem=problem,N=100,mutacion=.15,elitismo=
    # problem = datos del problema
    # N = Tamaño de la población
    # mutacion = probabilidad de una mutación
    # elitismo = porcion de la mejor poblacion a mantener
    # generaciones = nº de generaciones a generar para finalizar

    #Genera la poblacion inicial
    Nodos = list(problem.get_nodes())
    poblacion = generar_poblacion(Nodos,N)

    #Inicializamos valores para la mejor solucion
    (mejor_solucion, mejor_distancia) = Evaluar_Poblacion(poblacion,

    #Condicion de parada
    parar = False
    n=0
    #Iniciamos el ciclo de generaciones
    while(parar == False) :
```

```

#Cruce de la poblacion(incluye mutación)
poblacion = Cruzar(poblacion, mutacion, problem)

#Seleccionamos la población
poblacion = Seleccionar(problem,poblacion, N, elitismo)

#Evaluamos la nueva población
(mejor_solucion, mejor_distancia) = Evaluar_Poblacion(poblacion

print("Generacion #", n, "\nLa mejor solución es:" , mejor_solu

#Numero de generaciones. Criterio de parada
if n==generaciones:
    parar = True
    n +=1

return mejor_solucion

```

```
sol = algoritmo_genetico(problem=problem,N=500,mutacion=.3,elitismo
```

```

-----
-----
NameError                                Traceback (most
call last)
/tmp/ipykernel_4094/2726418425.py in <cell line: 0>()
      41
      42
--> 43 sol =
algoritmo_genetico(problem=problem,N=500,mutacion=.3,eliti

/tmp/ipykernel_4094/2726418425.py in algoritmo_genetico(pr
mutacion, elitismo, generaciones)
     10 #Genera la poblacion inicial

```

Pasos siguientes: [Explicar error](#)

